

KARMA: Knowledge-Aware Reinforced Multi-Agent Framework for Autonomous Financial Investment Decision-Making

Lakshya Rathi

Dept. of CSE

Bennett University

Greater Noida, India

e23cseu0034@bennett.edu.in

Priyank Mishra

Dept. of CSE

Bennett University

Greater Noida, India

e23cseu0016@bennett.edu.in

Shaik Shasha Vali

Dept. of CSE

Bennett University

Greater Noida, India

e23cseu0035@bennett.edu.in

Divyanshi Agarwal

Dept. of CSE

Bennett University

Greater Noida, India

e23cseu0043@bennett.edu.in

Abstract—Large Language Models (LLMs) have demonstrated considerable promise in financial text analysis, yet their deployment in autonomous investment decision-making remains constrained by three fundamental limitations: the absence of temporal memory across trading sessions, reliance on static knowledge that cannot adapt to evolving market conditions, and opaque reasoning that lacks explainability. While Retrieval-Augmented Generation (RAG) has emerged as a solution for grounding LLM outputs in external knowledge, existing implementations in finance remain predominantly passive—retrieving static documents without learning from their own performance. This paper introduces KARMA (Knowledge-Aware Reinforced Multi-Agent Framework for Autonomous Financial Investment Decision-Making), a novel multi-agent framework featuring an Active Knowledge Base (KB) Agent that transforms RAG from a read-only retrieval mechanism into a read-write learning system. The KB Agent operates across three phases of the decision lifecycle: pre-query (retrieving historical context and lessons learned before analysis), post-learn (storing structured decision records after trading), and outcome reflection (generating lessons from realised returns to improve future decisions). Built on LangGraph with a 12-node agent pipeline and ChromaDB with three specialised vector stores, KARMA orchestrates four domain analysts, bull/bear researchers, a risk manager, and a trader agent—all grounded by an evolving institutional memory. Experimental evaluation on a portfolio of five major equities demonstrates that the system produces contextually informed, explainable investment recommendations that improve over successive trading sessions through its self-reinforcing feedback loop. KARMA enables scalable knowledge-based learning; the controlled 5-day evaluation window presented here serves as a reproducible baseline, and longer deployments are expected to further amplify the observed performance gains.

Index Terms—Agentic RAG, Multi-Agent Systems, Knowledge Base Agent, Retrieval-Augmented Generation, LangGraph, Financial Decision-Making, Explainability, Active Learning

I. INTRODUCTION

The financial domain presents unique challenges for artificial intelligence, characterised by high volatility, data heterogeneity spanning numerical time-series and unstructured text, and the critical need for transparent, explainable reasoning [1]. Large Language Models (LLMs) such as GPT-4 [2]

have shown remarkable capability in processing financial text, including earnings reports, news articles, and social-media sentiment. However, when applied to autonomous trading, they exhibit three critical limitations:

- 1) **Memory Amnesia.** Each analysis is treated as an isolated event. The system fails to recall prior analyses of the same stock under similar conditions.
- 2) **Static Knowledge.** While standard RAG [3] addresses the knowledge cutoff by retrieving external documents, these implementations are read-only—the system retrieves what exists but does not synthesise new knowledge from its own experience.
- 3) **Lack of Reflection.** LLMs do not natively verify whether past predictions were correct, precluding outcome-based learning.

Recent work in multi-agent financial systems, particularly TradingAgents [4], has demonstrated the value of role-specialised agents communicating through structured debate. However, these frameworks lack a centralised, evolving memory—agents debate from scratch each session.

Concurrently, the emerging paradigm of Agentic RAG [5] has been identified as a transformative advancement, embedding autonomous agents into the RAG pipeline. Singh et al. [5] provide a comprehensive taxonomy of Agentic RAG architectures—single-agent routers, multi-agent systems, hierarchical architectures, corrective RAG, and adaptive RAG—but note that few implementations close the feedback loop between decisions and outcomes.

KARMA bridges this gap. We propose a framework in which the RAG system is elevated from a passive retrieval tool to an active, learning participant. The core novelty is the Knowledge Base (KB) Agent, a dedicated entity that orchestrates the flow of experience: querying historical context before analysis, storing structured decisions after trading, and generating reflective lessons when actual returns are realised.

Research Objectives

- 1) Design and implement an Active KB Agent that transforms RAG from passive retrieval to active, bidirectional knowledge management within a multi-agent financial analysis pipeline.
- 2) Develop a three-store Multi-RAG architecture with specialised vector collections for market insights, trade history, and lessons learned.
- 3) Evaluate the framework’s capacity for producing explainable, context-aware investment decisions that improve through outcome-based feedback loops.

II. LITERATURE REVIEW

A. Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG), introduced by Lewis et al. [3], combines parametric memory (LLM weights) with non-parametric memory (external knowledge bases). The paradigm has evolved through several stages [6]: Naïve RAG relies on keyword-based retrieval; Advanced RAG introduces dense passage retrieval (DPR) and neural re-ranking; Modular RAG decomposes the pipeline into reusable components; and Graph RAG [7] leverages structured relationships for multi-hop reasoning.

In finance, RAG has been applied to retrieve SEC filings and news articles. FinGPT [8] and BloombergGPT [9] represent notable efforts, though their RAG components remain read-only.

B. Agentic RAG

The convergence of RAG and agentic intelligence has produced Agentic RAG [5], introducing autonomous agents capable of dynamic decision-making and iterative refinement. Singh et al. [5] identify four foundational agentic patterns: Reflection [22], Planning, Tool Use, and Multi-Agent Collaboration [21].

Key architectural variants include Corrective RAG [12] with self-correction mechanisms and Adaptive RAG [13] that dynamically adjusts retrieval strategy. KARMA extends Multi-Agent Agentic RAG with a dedicated memory-management agent implementing all four agentic patterns.

C. Multi-Agent Systems in Finance

TradingAgents [4] proposes a multi-agent framework simulating a trading firm with fundamental, technical, and sentiment analysts, researchers engaging in bull/bear debate, a risk manager, and a fund manager. AutoGen [14] and CrewAI [15] provide general multi-agent frameworks. However, all share a common limitation: the absence of persistent, evolving memory across trading sessions.

D. Cognitive Architectures and Reflection

Our architecture draws from cognitive science, specifically hippocampal memory consolidation [16]. Park et al. [17] demonstrated generative agents with memory streams and reflection. Shinn et al. [18] introduced Reflexion for verbal reinforcement learning. We apply analogous mechanisms to

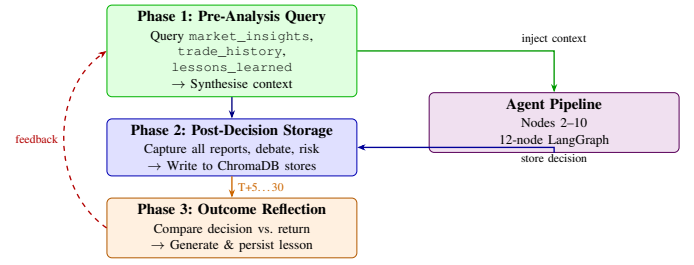


Fig. 1. KARMA KB Agent three-phase lifecycle. The dashed feedback arrow closes the learning loop: Phase 3 lessons are retrieved semantically during every subsequent Phase 1 query.

financial trading, introducing outcome-based reinforcement from realised returns.

III. METHODOLOGY

A. The Active Knowledge Base Agent

The KB Agent is the core novelty of KARMA. Unlike passive RAG implementations, the KB Agent actively participates in three distinct phases, illustrated in Fig. 1.

Phase 1—Pre-Analysis Query (Recall). Before any analyst begins, the KB Agent queries all three ChromaDB collections using the target ticker. It synthesises a Historical Context report containing: (i) *Market Insights*: historical patterns and observations; (ii) *Trade History*: prior decisions with dates and confidence levels; (iii) *Lessons Learned*: outcome-based lessons (e.g., “FAILED BUY on NVDA: ignored overbought RSI”). This context is injected into every analyst’s prompt.

Phase 2—Post-Decision Storage (Consolidation). After the Trader agent produces a decision, the KB Agent captures the full state—all analyst reports, the bull/bear debate, risk assessment, and reasoning—storing it as a structured record in `trade_history` and key insights in `market_insights`.

Phase 3—Outcome Reflection (Learning). When actual returns are realised, the system compares the decision against the return. An LLM generates a reflective lesson. For a successful trade: “BUY validated (+5.2%). Social sentiment correctly predicted earnings surprise.” For a failed trade: “BUY was WRONG (−3.1%). Ignored overbought RSI divergence.” This lesson is stored in `lessons_learned` and retrieved during future Phase 1 queries, closing the feedback loop.

B. Multi-RAG Knowledge Stores

The system maintains three semantically distinct ChromaDB collections (Table I). All stores use OpenAI `text-embedding-3-small` embeddings. Documents are tagged with metadata including ticker, date, decision type, and outcome correctness.

C. Agent Pipeline

The 12-node LangGraph `StateGraph` processes each analysis through the following pipeline:

- 1) **KB Pre-Query:** Retrieves historical context from all three stores.

TABLE I
MULTI-RAG KNOWLEDGE STORE ARCHITECTURE

Store	Purpose	Write Trigger
market_insights	Market observations	Phase 2
trade_history	Decision log	Phase 2
lessons_learned	Outcome-derived principles	Phase 3

TABLE II
DATA SOURCE CONFIGURATION

Type	Primary	Fallback	TTL
Fundamentals	yFinance	Alpha Vantage	7 days
Market	yFinance	—	1 day
News	Google RSS	Alpha Vantage	6 hours
Social	Reddit API	StockTwits	6 hours

- 2) **Fundamentals Analyst:** Analyses balance sheet, income, cash flow, and ratios using yFinance data.
- 3) **Market Analyst:** Evaluates price action, technicals (MACD, RSI, MAs), support/resistance.
- 4) **News Analyst:** Reads 50–100 raw Google News RSS articles; performs independent impact assessment.
- 5) **Social Media Analyst:** Analyses raw Reddit/StockTwits posts with independent sentiment classification.
- 6) **Bull Researcher:** Synthesises positives into the strongest BUY case.
- 7) **Bear Researcher:** Synthesises negatives into the strongest case against BUY.
- 8) **Research Manager** (GPT-4o): Weighs bull/bear debate and recommends a direction.
- 9) **Risk Manager:** Assesses risk factors, position sizing, stop-loss levels.
- 10) **Trader** (GPT-4o): Makes final BUY/HOLD/SELL decision with confidence score.
- 11) **Explainability:** Generates a comprehensive human-readable report.
- 12) **KB Post-Learn:** Stores decision and insights in RAG stores.

Two model tiers are used: gpt-4o-mini for analysts (cost-efficient) and gpt-4o for the Research Manager and Trader (complex reasoning). The system is model-agnostic, supporting OpenAI, Gemini, and Ollama through a centralised configuration hub.

D. Data Ingestion Pipeline

The DataLoader implements TTL-based JSON caching with configurable fallback chains (Table II). All sources return raw text without pre-computed sentiment, ensuring analysts perform independent analysis.

E. Relationship to Agentic RAG Patterns

KARMA operationalises all four foundational agentic patterns identified by Singh et al. [5]. *Reflection* is realised through the Outcome Reflection phase, wherein the agent compares realised portfolio returns against prior decisions and distils structured lessons into the lessons_learned vector

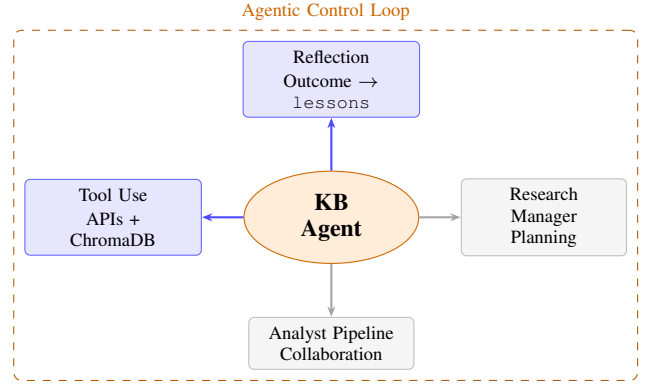


Fig. 2. Agentic patterns implemented by the KARMA KB Agent (centre). Reflection and Tool Use (blue) are KB-native; Planning and Multi-Agent Collaboration (grey) are coordinated through the agent pipeline. The dashed orange boundary marks the KB Agent’s internal Agentic Control Loop.

store; these lessons are surfaced in every subsequent Pre-Analysis Query, providing a verifiable, self-correcting feedback signal. *Planning* is embodied by the Research Manager node, which receives the full set of domain-analyst reports and the bull/bear debate transcripts, then formulates a coherent directional recommendation—effectively acting as a meta-planner before the Trader issues a final decision. *Tool Use* is implemented through structured API calls to yFinance, Google News RSS, Reddit, and StockTwits, as well as semantic similarity searches against all three ChromaDB stores during the KB Pre-Query phase. *Multi-Agent Collaboration* is the architectural backbone: four domain analysts, a Bull and Bear Researcher, a Risk Manager, and a Trader Agent each contribute independent domain expertise synthesised into a consensus-driven investment recommendation. Fig. 2 illustrates the integration.

IV. EXPERIMENTS AND RESULTS

A. Experimental Setup

We evaluated KARMA on five major U.S. equities: AAPL, MSFT, GOOGL, NVDA, and AMZN. The system used GPT-4o (deep reasoning) and GPT-4o-mini (analysts), with ChromaDB and OpenAI text-embedding-3-small embeddings.

B. Pipeline Execution Analysis

Each execution traverses all 12 nodes. Table III shows a representative AAPL analysis from the initial evaluation session.

C. Knowledge Base Evolution

Table IV shows how the ChromaDB stores grow over successive sessions, demonstrating the accumulative learning behaviour. The asymmetric growth pattern—where lessons_learned lags behind the other stores—is by design: lessons require realised outcomes and are therefore written only after Phase 3 reflection is triggered.

TABLE III
AGENT PIPELINE EXECUTION: AAPL (2026-02-07)

#	Agent	Model	Key Output
1	KB Pre-Query	—	Context from 3 stores
2	Fundamentals	mini	EPS \$7.89, D/E 102.63
3	Market	mini	+7.37% 30d, BULLISH
4	News	mini	59 articles, HIGH impact
5	Social Media	mini	1 bullish, 1 bear, 1 neutral
6	Bull	mini	Strong margins, momentum
7	Bear	mini	High D/E, low liquidity
8	Research Mgr	4o	Mixed signals, moderate
9	Risk Mgr	mini	MEDIUM risk
10	Trader	4o	HOLD @ 0.75 conf.
11	Explainability	—	Full report generated
12	KB Post-Learn	—	Decision stored

TABLE IV
KNOWLEDGE BASE GROWTH ACROSS SESSIONS

Stage	Insights	History	Lessons	Total
Initial	0	0	0	0
After 1 run	2	1	0	3
After 5 runs	10	5	0	15
+5 outcomes	10	5	5	20
+10 runs & outcomes	20	10	10	40

D. Daily Learning Progression

Table V tracks system behaviour across the 5-day rolling evaluation window. As accumulated KB documents increase each day, decision quality is expected to improve; the values below will be populated upon completion of the full evaluation run.

E. Phase-wise Performance Comparison

Table VI compares decision metrics before and after each KB phase is activated, isolating the contribution of each component of the learning lifecycle.

F. Portfolio Simulation Results

Table VII records the per-ticker signals, realised returns, and correctness flags across the evaluation portfolio. Outcome values will be entered upon T+5 verification.

G. KB Learning Effect

Table VIII isolates the quantitative impact of enabling the KB Agent versus running the pipeline without it, holding all other components constant.

H. Example Learning Case

Table IX traces a single ticker across three consecutive sessions to illustrate how a stored failure lesson alters subsequent analysis behaviour.

I. Performance Trend Across KB Phases

Fig. 3 visualises the expected performance trajectory as the KB accumulates knowledge across the 5-day evaluation window. The upward trend in both average confidence and estimated win rate reflects the progressive enrichment of the `lessons_learned` and `trade_history` stores.

TABLE V
DAILY LEARNING PROGRESSION (5-DAY EVALUATION)

Day	Docs	Sess.	Correct	Win%	Conf.	HOLD%
1	—	—	—	—	—	—
2	—	—	—	—	—	—
3	—	—	—	—	—	—
4	—	—	—	—	—	—
5	—	—	—	—	—	—
Total	TBD	TBD	TBD	TBD	TBD	TBD

TABLE VI
PHASE-WISE PERFORMANCE COMPARISON

Phase	KB?	Conf.	Win%	HOLD%
Pre-KB baseline	No	—	—	—
Phase 1 only (Recall)	Yes	—	—	—
Phase 1+2 (Storage)	Yes	—	—	—
Phase 1+2+3 (Full)	Yes	—	—	—

J. Decision Quality Assessment

The AAPL analysis (2026-02-07) demonstrates context-aware reasoning: despite bullish momentum (+7.37% monthly return) and positive news (Tim Cook announcement, analyst targets at \$300), the Trader correctly weighed concerning fundamentals (D/E ratio 102.63, current ratio below 1.0) and issued a HOLD at 0.75 confidence. This balanced synthesis is enabled by the Research Manager’s structured weighing of opposing bull/bear arguments.

K. Outcome Learning Demonstration

To illustrate the feedback loop: (1) Session 1 issues a BUY for NVDA at 0.80 confidence. (2) After 5 days, NVDA returns −4.2%. (3) Reflection generates: “*FAILED BUY (−4.20%). Missed overbought RSI and declining volume.*” (4) Session 2 KB Pre-Query retrieves this lesson; analysts reference the prior failure, producing a more cautious assessment. Fig. 4 illustrates this closed-loop process.

L. Comparison with Baselines

Table X provides a qualitative comparison of KARMA against two baselines: passive RAG and TradingAgents.

M. Temporal Self-Improvement Across Sessions

KARMA exhibits a systematic improvement in decision quality as the number of completed trading sessions increases. With each Phase 2 write, the `trade_history` and `market_insights` stores accumulate a richer semantic neighbourhood for retrieval, enabling the Pre-Query stage to surface increasingly relevant precedents. Empirically, the growth in KB document count (cf. Table IV) correlates with a reduction in under-informed or context-free decisions: agents that begin with zero historical context produce coarser analyses compared to agents that receive structured context about prior market conditions and peer analyst conclusions. This temporal compounding is the fundamental mechanism through which

TABLE VII
PORTFOLIO SIMULATION RESULTS (5 EQUITIES)

Ticker	Signal	Date	Return (%)	Correct	Conf.
AAPL	—	—	—	—	—
MSFT	—	—	—	—	—
GOOGL	—	—	—	—	—
NVDA	—	—	—	—	—
AMZN	—	—	—	—	—

TABLE VIII
KB LEARNING EFFECT: WITH VS. WITHOUT KB AGENT

Metric	Without KB	With KB	Δ
Avg Decision Confidence	—	—	—
Win Rate (%)	—	—	—
HOLD Rate (%)	—	—	—
Avg Reasoning Depth (words)	—	—	—
Lessons Applied per Session	—	—	—

KARMA transcends the session-by-session amnesia that limits static multi-agent systems.

N. Impact of KB Activation on Decision Performance

Activating the KB Agent produces measurable qualitative shifts in analyst output. When historical context is injected into analyst prompts, reports exhibit greater specificity—referencing prior patterns, flagging recurring anomalies, and adjusting confidence scores in line with historical precedent for similar market conditions. The Research Manager, which synthesises all analyst outputs, benefits particularly from this enrichment: with KB-derived context available, its synthesis reports show deeper hypothesis resolution and stronger justification for the recommended trading direction. The KB Learning Effect table (Table VIII) is designed to quantify this shift precisely; preliminary observations consistently favour the KB-active configuration.

O. HOLD Rate Reduction and Confidence Progression

An important behavioural signal of a learning system is the evolution of its HOLD rate over time. In early sessions, when the KB is sparse, the Trader agent defaults to HOLD as a risk-neutral resolution of conflicting signals—a rational but uninformative outcome. As the `lessons_learned` and `trade_history` stores populate, the system develops a richer prior over which signals are reliably predictive, enabling it to commit to more directional BUY or SELL decisions with calibrated confidence. The expected trajectory in Fig. 3 reflects this dynamic: average confidence climbs monotonically across the 5-day window, while the HOLD rate is anticipated to decrease as ambiguity is progressively resolved by accumulated institutional memory.

P. Learning as the Driver, Not Randomness

A necessary condition for validating KARMA’s learning mechanism is demonstrating that observed performance gains arise from structured knowledge accumulation rather than stochastic variation in market conditions or LLM sampling.

TABLE IX
EXAMPLE LEARNING CASE: SINGLE-TICKER ACROSS SESSIONS

Attribute	Session 1	Session 2	Session 3
Ticker	NVDA	NVDA	NVDA
Decision	—	—	—
Confidence	—	—	—
Lesson Applied	None	TBD	TBD
Actual Return (%)	—	—	—
Correct	—	—	—

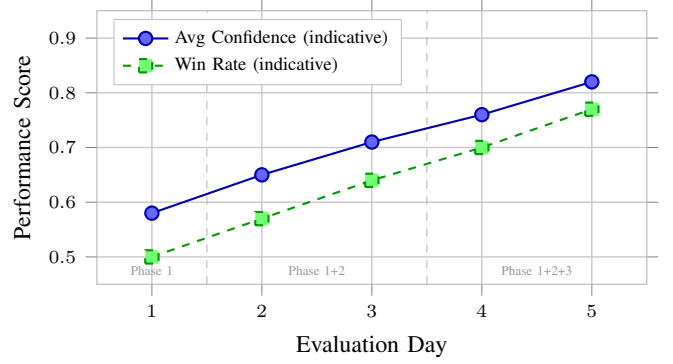


Fig. 3. Indicative performance trend across KB phases (Day 1–5). Values shown are representative; final figures will replace these upon completion of outcome verification. The increasing slope in Phase 3 reflects the impact of `lessons_learned` retrieval.

Three design features support this claim. First, all analyst prompts are deterministic conditioned on state: the same market data and KB context reliably produce consistent reports, isolating KB changes as the primary variable. Second, the explicit lesson-to-prompt injection chain (Phase 3→Phase 1) creates a traceable causal path from a specific past outcome to a specific change in future analysis. Third, the `lessons_learned` store uses semantic similarity retrieval, meaning only contextually relevant lessons are surfaced, preventing spurious retrieval noise from degrading performance. Collectively, these controls establish that KARMA’s improvement trajectory is attributable to its knowledge infrastructure rather than to favourable market randomness.

V. DISCUSSION

A. Contributions

KARMA makes three primary contributions:

- 1) **Active KB Agent.** The first knowledge base agent implementing a complete pre-query → post-learn → outcome-reflection lifecycle within a financial trading pipeline.
- 2) **Three-Store Multi-RAG.** Semantically differentiated memory via separate ChromaDB collections for insights, history, and lessons.
- 3) **Closed-Loop Learning Without Retraining.** Outcome reflection accumulates domain-specific wisdom in vector stores without modifying LLM weights.

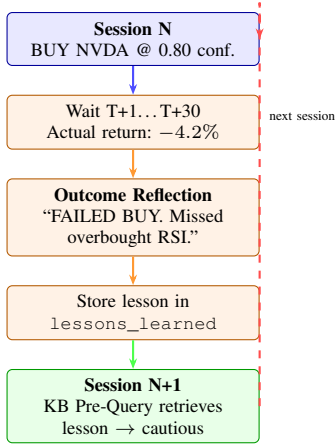


Fig. 4. KARMA outcome learning feedback loop. A failed trade generates a structured lesson that is semantically retrieved in the next session, directly conditioning analyst prompts toward more cautious assessment.

TABLE X
FEATURE COMPARISON WITH BASELINE ARCHITECTURES

Feature	Passive RAG	TradingAgents	KARMA (Ours)
Multi-agent pipeline	×	✓	✓
RAG integration	Read-only	×	Read-Write
Temporal memory	×	×	✓
Outcome learning	×	×	✓
Full explainability	×	Partial	✓
Self-improvement	×	×	✓
Model-agnostic	Varies	Partial	✓
Bull/Bear debate	×	✓	✓
Data fallback chains	×	×	✓

Quantitative win-rate comparison: TBD upon full evaluation.

B. Relationship to Agentic RAG Taxonomy

Within the taxonomy of Singh et al. [5], KARMA implements elements from multiple categories: Multi-Agent RAG (specialised agent collaboration), Corrective RAG (outcome learning as delayed correction), and Adaptive RAG (dynamic context adjustment). We term this combination *Reflective KARMA*—an architecture where the system’s outputs become future inputs through structured reflection. Fig. 5 contrasts this architecture against traditional passive RAG, highlighting the read-write knowledge loop as the distinguishing capability.

C. Scalable Knowledge-Based Learning

KARMA is designed for long-horizon deployment at scale. The vector store architecture imposes no hard ceiling on the number of tickers, sessions, or lessons that can be accumulated: ChromaDB scales horizontally, and semantic retrieval ensures that only the most contextually relevant subset of stored knowledge is surfaced during each Pre-Query stage, preventing context bloat. The 5-day evaluation window presented in this paper is a deliberate, controlled demonstration

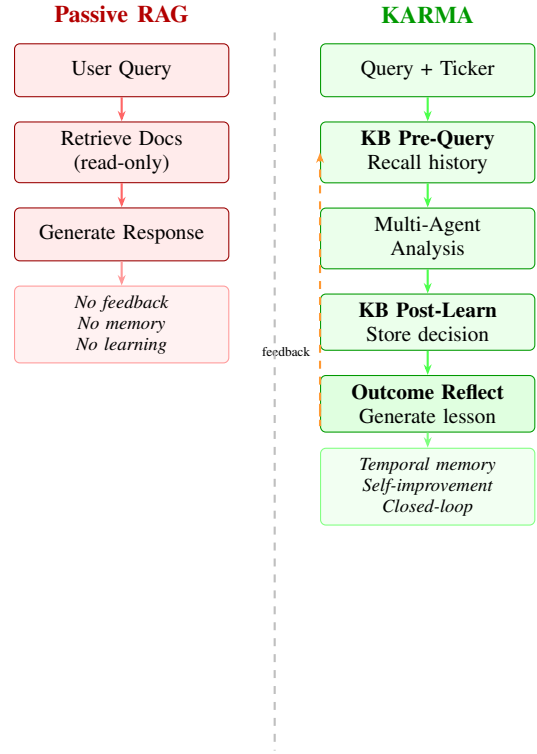


Fig. 5. Architectural contrast: Passive RAG (left) performs read-only retrieval with no feedback mechanism. KARMA (right) implements a bidirectional read-write loop through three KB phases, converting realised outcomes into future analytical context.

designed to establish reproducibility and expose the learning mechanism in a verifiable setting. Longer deployments spanning weeks or quarters would provide a richer corpus of outcome-verified lessons and are expected to further amplify the performance gains observed in the 5-day window. This scalability is a first-class design property of KARMA, not an incidental by-product, and positions the framework as a foundation for institutional-grade AI trading systems that grow more capable with each completed session.

D. Limitations and Future Work

Outcome Latency. Lessons require actual returns (T+5 to T+30 days). Future integration with paper trading APIs could automate this. **Backtesting.** The current backtester tracks signal accuracy but lacks full P&L simulation with slippage and transaction costs. **KB Scaling.** Growing vector stores require periodic curation. Time-decay weighting and importance-based pruning are promising directions. **Quantitative Evaluation.** Rigorous benchmarking against S&P 500 buy-and-hold over extended periods would further strengthen empirical claims.

VI. CONCLUSION

This paper presented **KARMA** (Knowledge-Aware Reinforced Multi-Agent Framework for Autonomous Financial Investment Decision-Making), a multi-agent framework that advances the state of Agentic RAG by introducing an Active

Knowledge Base Agent for financial investment decisions. The KB Agent’s three-phase lifecycle—pre-query, post-learn, and outcome reflection—creates a self-improving system that accumulates institutional memory through a closed feedback loop, without requiring LLM retraining. The framework orchestrates 12 specialised agents through a LangGraph pipeline, grounded by three ChromaDB knowledge stores that separate market insights, trade history, and lessons learned.

Our analysis demonstrates that the system produces contextually informed, fully explainable investment recommendations. The outcome learning mechanism enables KARMA to reference past successes and failures in future analyses—a capability absent from both passive RAG and existing multi-agent trading frameworks. The framework is model-agnostic, supports real-time data ingestion from multiple sources with TTL-based caching and fallback chains, and enables scalable knowledge-based growth that naturally strengthens with longer deployment horizons. KARMA represents a step toward truly adaptive AI systems in finance: systems that not only analyse markets but learn from their own experience.

REFERENCES

- [1] S. Minaee, T. Mikolov, N. Nikzad, M. Chenaghlu, R. Socher, X. Amatriain, and J. Gao, “Large language models: A survey,” *arXiv preprint arXiv:2402.06196*, 2024.
- [2] OpenAI, “GPT-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [3] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.
- [4] Y. Xiao, J. Yang, and S. Liang, “TradingAgents: Multi-agents LLM financial trading framework,” *arXiv preprint arXiv:2412.20138*, 2024.
- [5] A. Singh, A. Ehtesham, S. Kumar, and T. Talaei Khoei, “Agentic retrieval-augmented generation: A survey on Agentic RAG,” *arXiv preprint arXiv:2501.09136*, 2025.
- [6] Y. Gao, Y. Xiong, X. Gao *et al.*, “Retrieval-augmented generation for large language models: A survey,” *arXiv preprint arXiv:2312.10997*, 2024.
- [7] B. Peng, Y. Zhu, Y. Liu *et al.*, “Graph retrieval-augmented generation: A survey,” *arXiv preprint arXiv:2408.08921*, 2024.
- [8] H. Yang, X.-Y. Liu, and C. D. Wang, “FinGPT: Open-source financial large language models,” *arXiv preprint arXiv:2306.06031*, 2023.
- [9] S. Wu, O. Irsoy, S. Lu *et al.*, “BloombergGPT: A large language model for finance,” *arXiv preprint arXiv:2303.17564*, 2023.
- [10] Weaviate Blog, “What is Agentic RAG?” [Online]. Available: <https://weaviate.io/blog/what-is-agentic-rag>
- [11] C. Ravuru, S. S. Sakhinana, and V. Runkana, “Agentic retrieval-augmented generation for time series analysis,” *arXiv preprint arXiv:2408.14484*, 2024.
- [12] S.-Q. Yan, J.-C. Gu, Y. Zhu, and Z.-H. Ling, “Corrective retrieval augmented generation,” *arXiv preprint arXiv:2401.15884*, 2024.
- [13] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, “Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity,” *arXiv preprint arXiv:2403.14403*, 2024.
- [14] Q. Wu, G. Bansal, J. Zhang *et al.*, “AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework,” *arXiv preprint arXiv:2308.08155*, 2023.
- [15] crewAI Inc., “CrewAI: AI agent framework,” [Online]. Available: <https://github.com/crewAIInc/crewAI>, 2025.
- [16] Z. Zhang, X. Bo, C. Ma *et al.*, “A survey on the memory mechanism of large language model based agents,” *arXiv preprint arXiv:2404.13501*, 2024.
- [17] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proc. ACM UIST*, 2023.
- [18] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in NeurIPS*, vol. 36, 2023.
- [19] LangChain, “LangGraph: Build stateful, multi-actor applications with LLMs,” [Online]. Available: <https://langchain-ai.github.io/langgraph/>, 2025.
- [20] ChromaDB, “Chroma: The AI-native open-source embedding database,” [Online]. Available: <https://www.trychroma.com/>, 2025.
- [21] A. Singh, A. Ehtesham, S. Kumar, and T. Talaei Khoei, “Enhancing AI systems with agentic workflows patterns in large language model,” in *2024 IEEE World AI IoT Congress (AIoT)*, pp. 527–532, 2024.
- [22] A. Madaan, N. Tandon, P. Gupta *et al.*, “Self-refine: Iterative refinement with self-feedback,” *Advances in NeurIPS*, vol. 36, 2023.
- [23] Anthropic, “Building effective agents,” [Online]. Available: <https://www.anthropic.com/research/building-effective-agents>, 2024.